

Final Project Report

1. Experience with LLM-based Hardware Design

In this project, I used a Large Language Model (LLM) to generate Verilog implementations of arithmetic circuits, including an 8-bit ripple-carry adder (RCA8) and a Kogge-Stone adder (KSA8). Overall, the experience demonstrates that LLMs are effective tools for accelerating hardware design, especially for well-understood and structured circuits.

For simple designs such as RCA8, the LLM was able to quickly generate structurally correct and synthesizable Verilog code with minimal manual intervention. The generated code could be directly used in synthesis and simulation flows, significantly reducing development time.

However, for more complex architectures such as KSA8, the LLM-generated code often required careful inspection and manual correction. Issues such as incorrect signal indexing, missing intermediate signals, or inconsistent module definitions were more common. This indicates that while LLMs are useful, they are not yet fully reliable for complex digital designs without human verification.

More broadly, this project suggests that LLMs in chip design (LLM4ChipDesign) have not yet reached a stage where they can fully replace human designers. Instead, they currently function best as productivity-enhancing assistants, helping engineers generate initial designs, debug errors, and iterate faster.

2. Accuracy and Limitations of LLM-generated Code

The accuracy of LLM-generated code strongly depends on the complexity of the design:

- High accuracy for simple designs
The RCA8 design was generated correctly and passed both functional simulation and formal equivalence checking. This shows that LLMs perform well for standard, repetitive structures.
- Reduced reliability for complex designs
For KSA8, the generated code exhibited several issues:
 - Incorrect or incomplete prefix logic
 - Signal indexing errors
 - Missing or inconsistent internal connections

These errors required manual debugging and correction. Therefore, LLM-generated

code should not be treated as ground truth, especially for advanced architectures.

3. Quality of LLM-generated Testbenches

The LLM-assisted testbench generation pipeline was generally effective, particularly for simple designs:

- For RCA8, the generated testbench and golden model worked correctly and were able to validate the design successfully.
- The test patterns covered a reasonable range of input cases, including edge cases.

However, for KSA8, the generated Python golden model contained critical errors:

- It treated integer inputs as bit arrays (e.g., using `a[i]`), leading to runtime errors
- As a result, golden outputs could not be computed initially.

This highlights a key limitation:

LLMs may generate structurally plausible but semantically incorrect reference models.

Manual correction (replacing the golden model with a simple arithmetic reference) was required to restore correctness.

4. Lessons Learned and Best Practices

From this project, several important lessons were learned:

- 1) LLMs are best used as assistants, not replacements
They can generate a strong starting point but still require expert verification.
- 2) Always validate generated designs
Both simulation and formal equivalence checking are essential steps.
- 3) Keep golden models simple and reliable
Instead of reproducing complex hardware logic, use mathematical definitions when possible.
- 4) Check intermediate artifacts
Errors may not appear in final outputs but exist in intermediate steps (e.g., golden model generation).
- 5) Prompt quality matters
Clear constraints (module names, signal ranges, structural requirements) significantly improve output quality.

5. Recommendations for Using LLMs in Hardware Design

Based on this project, the following recommendations are suggested:

1. Use LLMs for rapid prototyping of simple designs
They are highly efficient for standard modules like adders, multiplexers, and simple controllers.
2. Avoid relying on LLMs for complex architectures without verification
Designs such as prefix adders require manual review and correction.
3. Integrate LLMs into a verification-driven workflow
Always pair LLM-generated code with: Simulation and Formal equivalence checking
4. Prefer simple golden models over structural ones
Mathematical correctness is more reliable than structural replication.
5. Use LLM outputs as a baseline, not final design
Human expertise remains essential for ensuring correctness, especially in complex digital systems.

In summary, LLMs significantly improve design efficiency by accelerating code generation, assisting with debugging, and reducing development time. However, they are not yet capable of fully autonomous hardware design.

Effective use of LLMs in chip design requires a human-in-the-loop approach, where engineers guide, verify, and refine the generated results.